

OOPs (Object-Oriented Programming) in Python – Notes

◇ 1. What is OOP?

- **Definition:** Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects** (data + methods).
- **Why?**
 - Reusability of code (through classes & inheritance)
 - Organizing large projects
 - Data hiding and security (encapsulation)
 - Real-world modeling (objects represent entities)

◇ 2. Basic OOP Terminologies

1. **Class** → Blueprint for creating objects (defines attributes & methods).

```
class Car:  
    def __init__(self, brand, color):  
        self.brand = brand  
        self.color = color
```

2. **Object** → Instance of a class.

```
car1 = Car("Tesla", "Red")
```

3. **Constructor** (`__init__`) → Special method called automatically when an object is created.
4. **Self** → Refers to the current object (used to access instance variables).

◇ 3. Four Pillars of OOP in Python

☑ 1. Encapsulation

- **Definition:** Wrapping variables (data) and methods (functions) into a single unit (class).
- **Where used:** Data hiding and restricting direct access.
- **Example:**

```
class Student:
    def __init__(self, name, age):
        self.__name = name      # private variable
        self.__age = age

    def get_age(self):
        return self.__age
```

👉 Here, __age is private (not directly accessible).

☑ 2. Inheritance

- **Definition:** One class can acquire properties and methods of another class.
- **Types in Python:**
 - **Single Inheritance**
 - **Multiple Inheritance**
 - **Multilevel Inheritance**
 - **Hierarchical Inheritance**
 - **Hybrid Inheritance**
- **Example:**

```
class Parent:
    def display(self):
        print("This is Parent class")
```

```
class Child(Parent):    # Inheriting Parent
    def show(self):
        print("This is Child class")

obj = Child()
obj.display()
obj.show()
```

3. Polymorphism

- **Definition:** "One name, many forms". The same method can behave differently in different situations.
- **Where used:** Function overloading, method overriding, operator overloading.
- **Example (Method Overriding):**

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

obj = Dog()
obj.sound()    # Output: Bark
```

- **Example (Operator Overloading):**

```
class Book:
    def __init__(self, pages):
        self.pages = pages

    def __add__(self, other):
        return self.pages + other.pages
```

```
b1 = Book(100)
b2 = Book(200)
print(b1 + b2) # Output: 300
```

☑ 4. Abstraction

- **Definition:** Hiding the implementation details and showing only the essential features.
- **Where used:** Abstract classes & interfaces.
- **In Python:** Implemented using abc (Abstract Base Class).
- **Example:**

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
```

```
class Car(Vehicle):
    def start(self):
        print("Car started")
```

```
obj = Car()
obj.start()
```

◇ 4. Key OOP Concepts in Python

1. Method Types

- a. **Instance Method** → Works with object (self).
- b. **Class Method** → Works with class (@classmethod, cls).
- c. **Static Method** → Independent of class & object (@staticmethod).

```

class Demo:
    def instance_method(self): # Instance Method
        print("Instance method")

    @classmethod
    def class_method(cls):     # Class Method
        print("Class method")

    @staticmethod
    def static_method():       # Static Method
        print("Static method")

```

2. Access Modifiers

- a. Public: var
- b. Protected: `_var`
- c. Private: `__var`

3. Duck Typing in Python

- a. If an object behaves like a type, it can be treated as that type.

```

class Bird:
    def fly(self): print("Flying")
class Airplane:
    def fly(self): print("Airplane flying")

def lift_off(obj):
    obj.fly()

lift_off(Bird())      # works
lift_off(Airplane()) # works

```

◇ 5. Advantages of OOP

- Code reusability
- Data security
- Modularity

- Scalability
- Easy to debug and maintain